IMX Model Mapping

Geonovum Standard
Editor's draft May 18, 2026

Latest editor's draft:
https://geonovum.github.io/IMX-ModelMapping/

Editor:
Paul Janssen (Geonovum)

Author:
Pano Maria (Geonovum)

Participate:
GitHub Geonovum/IMX-ModelMapping
File an issue
Commit history
Pull requests

This document is also available in these non-normative format: pdf

Abstract

This document specifies the IMX model mapping. The IMX model mapping expresses a mapping between one or more source data models, to a target data model, and is designed to be used as configuration for an IMX orchestration process.

Status of this document

This is a working draft that can be changed, removed or replaced by other documents at any time. It is not a stable document.

Table of Contents

§ 1. Introduction

This section is non-normative.

This document describes a proposed standard for mapping logical source data models to a logical target data model, in the context of IMX.

The IMX model mapping standard plays an important role in the reference architecture of the IMX orchestration process. The mapping serves as a formal translation syntax from one or more source models based API data sources towards one a target model based API data source. This mapping is at the basis of the actual IMX orchestration process.

Chapter 4: Model representation specifies the model to which a source model must be mapped to be able to be put into the IMX orchestration engine. Basically a model consists of objecttypes and properties. Filter mapping specify how source object are selected to match target objects according to defined filter operations/queries.

Chapter 5: Mapping abstract model implements the model defined in chapter 4 into a model for mapping between a source - and a target model containing all the necessary mapping rules. Objecttype mappping, path mapping (navigating through a source model) and property mapping are the basic concepts. Operators as result combiner, result mapper and matcher allow for defining complex combinations of chained results.

Chapter 6: YAML specification presents the YAML encoding of the abstract mapping model. This YAML IMX Mapping specification serves as a specification for YAML instance documents into the IMX Orchestration engine for actual operating on a set of source and target API's.

In annex 1 the mapping specifications of the IMX-Geo model are presented. These are presented in an excel spreadsheet and serve as input for the machine readable YAML formalisation presented in this document.

§ 2. Design Principles

This section is non-normative.

The mapping is implementation independent

An important design principle is that the mapping should be independent of technical implementations, data format, or data modeling standards. This allows for broad use of the mapping across disparate data sources and environments.

The mapping is declarative

The mapping should be declarative. By declarative we mean that the mapping is described in a high level abstract language, describing the intent of the mapping, without explicit procedural logic. This affords various different usage scenario's, like employing different procedural strategies during the execution of the mapping, while keeping the mapping simple to express.

The mapping is expressed in terms of logical models

In order to be independent of technical data models and corresponding data formats, the mapping should be expressed at the logical level. A logical data model establishes the structure of data items, and their relationships, yet is independent of their technical implementation. This contributes to the mappings implementation independence.

> NOTE
> Note that the logical model should be expressive enough to cover all its technical implementations. That is, it should be possible to translate expressions about data at the logical level, to the implementing technical level.

The mapping is independent of data modeling standards

Their exist a variety of different data modeling standards, all with their own intricacies in the way they define data items. The mapping should be independent of these. This will allow for broad applicability of the mapping.

§ 3. Basic concepts

This section is non-normative.

The IMX model mapping plays an important role in the reference architecture of the IMX orchestration process. The mapping serves as a translation from one or more source model based API data sources towards a target model based API data source.

The IMX model mapping is the core of what drives the orchestration process that will handle requests to the target API.

The ***IMX orchestration process*** is an activity which can answer requests on a target data source, generated in accordance with a target model, by orchestrating requests on the underlying source data sources, which correspond with source models, driven by a IMX model mapping, combining the results.

The orchestration process is carried out by an IMX orchestration engine. An ***IMX orchestration engine*** is a software component which takes an IMX model mapping, one or more source models

and source data sources, and a target model, and generates a target data source in accordance with the target model which carries out he orchestration process.

A ***data source*** is an entity which provides access to a dataset.

A ***source data source*** is a data source which acts as a source in an orchestration process.

A ***target data source*** is a data source which acts as the target in an orchestration process.

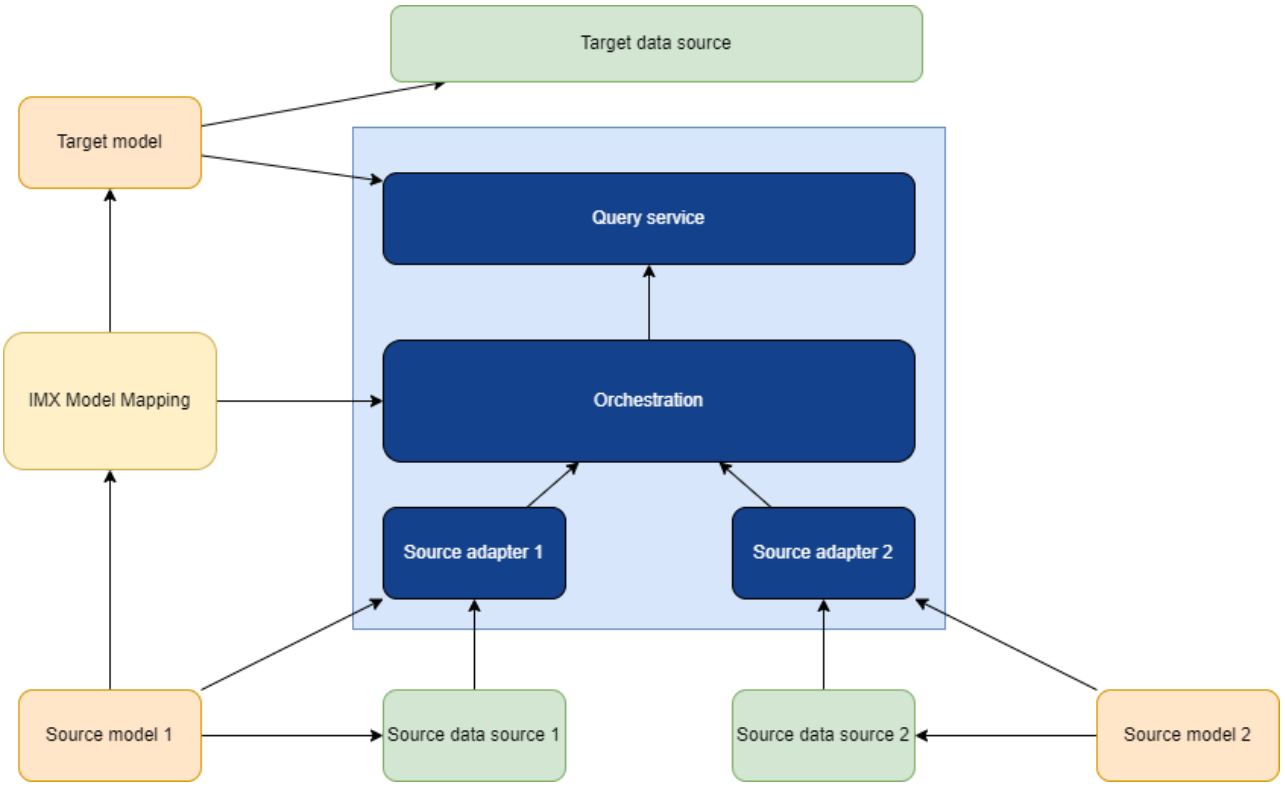

Figure 1 IMX Orchestration

The default way of expressing a model mapping is in [*YAML*] format. In recent years, [*YAML*] has emerged as a concise, readable format to represent information, which is widely applied as a configuration file format.

EXAMPLE 1: IMX Model Mapping in YAML

```yaml
objectTypeMappings:
  Address:
    - sourceRoot: bag:Nummeraanduiding
      propertyMappings:
        addressID:
          pathMappings:
            path: identificatie
        locatorDesignator:
          pathMappings:
            - path: huisnummer
            - path: huisnummertoevoeging
              map:
                type: prepend
                options:
                  prefix: ' '
            - path: huisletter
              map:
                type: prepend
                options:
                  prefix: ' '
          combine:
            type: join
        postCode:
          pathMappings:
            path: postcode
        postName:
          pathMappings:
            path: ligtIn/naam
            andThen:
              ifMatch:
                type: isNull
              path: ligtAan/ligtIn/naam
        thoroughfare:
          pathMappings:
            path: ligtAan/naam
```

§ 4. Representing models for orchestration

To make the IMX mapping and orchestration independent of underlying modeling standards and implementations, a simple internal logical data model representation is used, to which a source model must be mapped.

> NOTE
> The mapping to the IMX internal logical data model is left as an implementation detail.

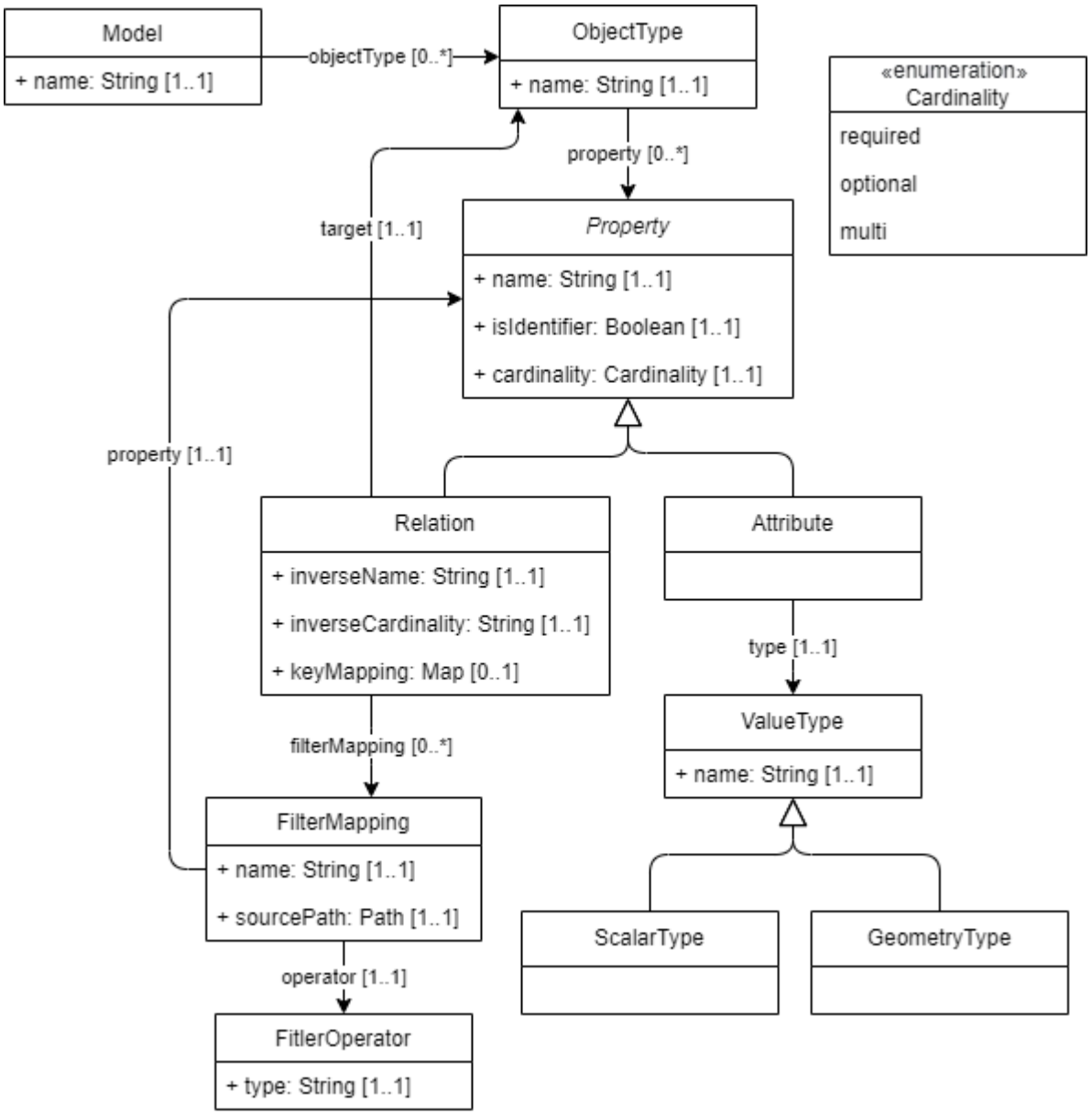

Figure 2 Model representation for orchestration

§ 4.1 Model (`Model`)

A ***model*** is the representation of a logical data model which can be used in an IMX orchestration engine.

A logical data model is used to define data items which which are used to describe objects.

A data item consists of a subject, a property and a value, which together represent an elementary statement about an object.

An ***object*** is anything that is the subject of a data item. A contextually grouped set of data items about the same object forms a ***data object***.

Overview attributes

Name	Multiplicity	Definition
name	1..1	The name of the model.

Overview relations

Name	Multiplicity	Definition
objectType	0..*	A relation pointing to an object type that is part of the model.

§ 4.2 Object type (`ObjectType`)

An ***object type*** represents the set of a type of object.

Overview attributes

Name	Multiplicity	Definition
name	1..1	The name of the object type.

Overview relations

Name	Multiplicity	Definition
property	0..*	A relation pointing to a property.
identityProperty	0..*	A relation pointing to a property which is identifying for the property-bearing object type.

§ 4.3 Filter mapping (`FilterMapping`)

A ***filter mapping*** speficies how to filter source objects matching the target object.

Overview relations

Name	Multiplicity	Definition
property	0..*	A relation pointing to a target object's property.
operator	0..*	A relation pointing to an operator which expresses a filter operation on the target object's property .
sourcePath	0..*	A relation pointing to a path on the objects of the source object type.

§ 4.4 Filter Operator (`Filter Operator`)

A ***filter operator*** specifies which operation is to be used to filter the matching source objects.

Overview attributes

Name	Multiplicity	Definition
type	0..*	The operator type .

§ 4.5 Property (`Property`)

A ***property*** is a predicate which is used to express a data item about an object.

Overview attributes

Name	Multiplicity	Definition
name	1..1	The name of the property.
isIdentifier	1..1	Indication whether the property is identifying.
multiplicity	1..1	The multiplicity of the property.

§ 4.5.1 Multiplicity (`Multiplicity`)

A ***multiplicity*** is an enumeration item which is used to express how many times a data item with the same property, and the same subject, can be expected to occur.

Overview enumeration items

Name	Definition
requiredOnce	Occurs exactly once.
requiredMulti	Occurs multiple times.
optionalOnce	Can occur at most once.
optionalMulti	Can occur multiple times.

§ 4.5.2 Relation (`Relation`)

A ***relation*** is a property which expresses a relationship between the relation-bearing object and a target object. It is a sub-type of Property.

Overview attributes

Name	Multiplicity	Definition
inverseName	1..1	The inverse name of the relation, to be used to traverse the relation in the inverse direction.
inverseMultiplicity	1..1	The inverse multiplicity of the relation.
keyMapping	0..1	A map (String -> Object) defining the mapping the source object type key properties to the target key properties.

Overview relations

Name	Multiplicity	Definition
target	0..*	The object type that is the target of the relation.
filterMapping	0..*	A relation pointing to a filter mapping to filter the matching source objects.

§ 4.5.3 Attribute (`Attribute`)

An ***attribute*** is a property which expresses a characteristic about an object.

Overview relations

Name	Multiplicity	Definition
type	1..1	The value type that is the type of the value of the data item expressing the attribute.

§ 4.6 Value type (`ValueType`)

A ***value type*** is a type of an attribute-expressing data item.

Overview attributes

Name	Multiplicity	Definition
name	1..1	The name of the value type.

§ 4.6.1 Scalar type (`ScalarType`)

A ***scalar type*** is a value type which is a scalar.

Common examples of scalars are:

- String
- Integer
- Boolean

§ 4.6.2 Geometry type (`GeometryType`)

A ***geometry type*** is a value type which represents a geometry.

§ 5. Mapping abstract model

The following model implements the model defined in chapter 4 into a model containing all the necessary mapping rules. Objecttype mappping, path mapping (navigating through a source model) and property mapping are the basic concepts. Operators as result combiner, result mapper and matcher allow for defining complex combinations of chained results.

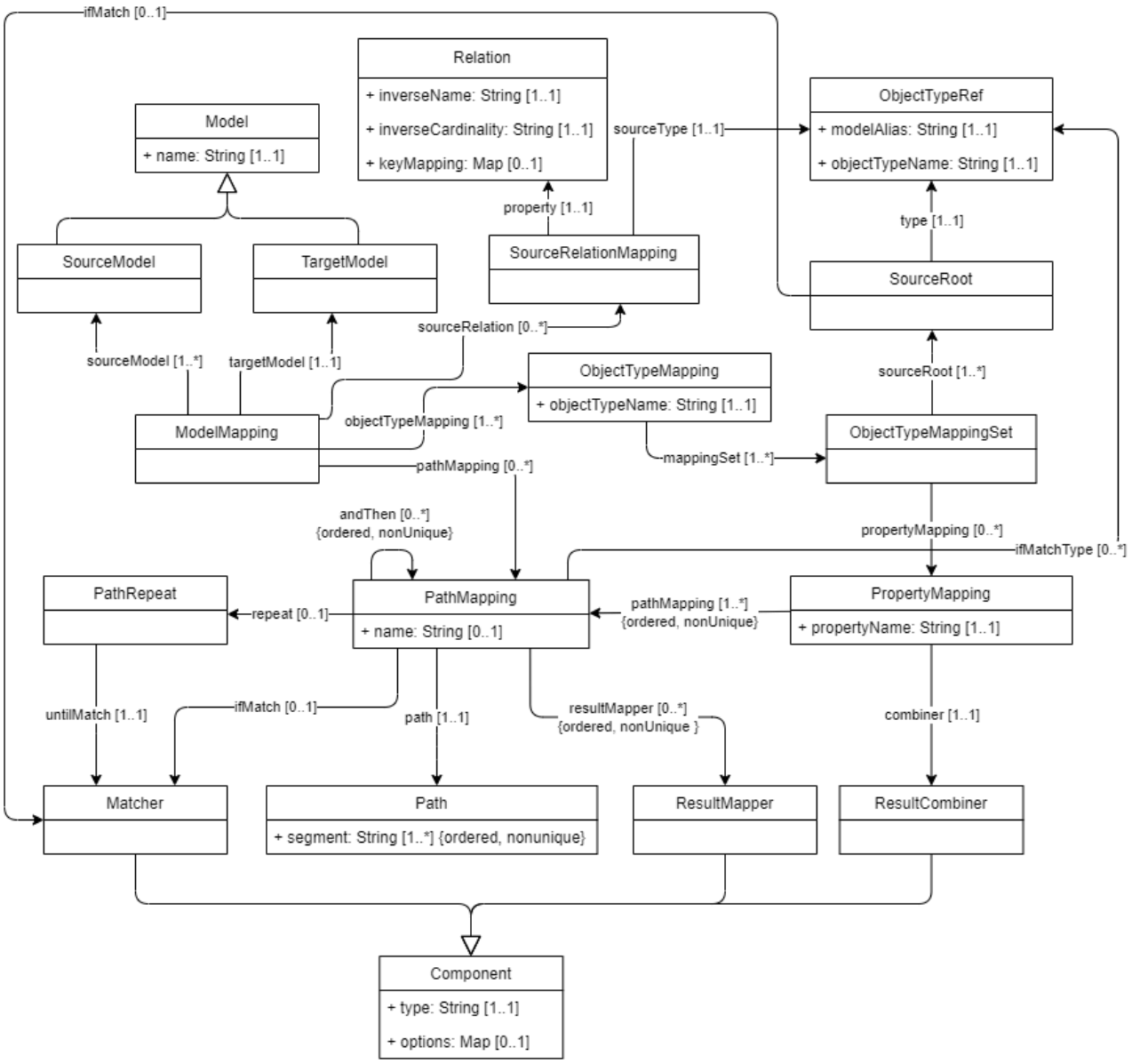

Figure 3 IMX Model Mapping

§ 5.1 Source model (`SourceModel`)

A ***source model*** is a model which is used to define the structure of data items in a source data source in an IMX orchestration process.

§ 5.2 Target model (`TargetModel`)

A ***target model*** is a model which is used to define the structure of data items in the target data source in an IMX orchestration process.

§ 5.3 Model mapping (`ModelMapping`)

A ***model mapping*** is a set of declarations which afford the combination and translation of one ore more source data model based data sources to a target data model based data source.

Overview relations

Name	Multiplicity	Definition
targetModel	1..1	A relation pointing to a target model.
sourceModel	0..*	A relation pointing to a source model.
sourceRelation	0..*	A relation pointing to a source relation mapping.
objectTypeMapping	1..*	A relation pointing to an object type mapping.
pathMapping	0..*	A relation pointing to a reusable named path mapping.

§ 5.4 Object type ref (`ObjectTypeRef`)

An ***object type ref*** represents a reference to an object type. A reference is expressed using a model alias and the name of an object type.

A ***model alias*** is a string of characters that uniquely identifies a model in an IMX orchestration process.

Overview attributes

Name	Multiplicity	Definition
modelAlias	1..1	The alias of a source model.
objectTypeName	1..1	The name of the object type.

§ 5.5 Source root (`SourceRoot`)

A ***source root*** is a set of declarations which express the starting point for path evaluations.

Overview relations

Name	Multiplicity	Definition
type	1..1	A relation pointing to a object type ref.

§ 5.6 Source relation mapping (`SourceRelationMapping`)

A ***source relation mapping*** is a set of declarations which define a virtual relation between object types in one or more source models. Virtual, in that the relation does not actually exist in the source models, but exists for the mapping process, through this mapping.

A relation defined through a source relation mapping can be used just as any other relation within object type mappings.

Overview relations

Name	Multiplicity	Definition
sourceType	1..1	A relation pointing to a object type ref, indicating the source object type of the relation.
property	1..1	A relation pointing to a newly defined relation.

§ 5.7 Object type mapping (`ObjectTypeMapping`)

An ***object type mapping*** is a set of declarations which express the mapping of one or more source model based data items using its object type mapping sets.

Overview attributes

Name	Multiplicity	Definition
objectTypeName	0..*	The name of the object type in the target model which is the target of this object type mapping.

Overview relations

Name	Multiplicity	Definition
mappingSet	1..*	A relation pointing to a object type mapping set.

§ 5.8 Object type mapping set (`ObjectTypeMappingSet`)

An ***object type mapping set*** is a set of declarations which express the mapping of one or more data items using a specicic source root to a target-model-object-type-based data object.

Overview relations

Name	Multiplicity	Definition
sourceModel	1..*	A relation pointing to a source model.
targetModel	1..1	A relation pointing to a target model.
sourceRoot	1..1	A relation pointing to a object type ref.
propertyMapping	0..*	A relation pointing to a property mapping.

§ 5.9 Property mapping (`PropertyMapping`)

A ***property mapping*** is a set of declarations which express the mapping of a target object type property within an object type mapping.

A property mapping can be expressed through a sinlge path mapping, or can be the result of a combination of path mappings by applying its specified combiner.

TODO: default combiner??

Overview attributes

Name	Multiplicity	Definition
propertyName	0..*	The name of the property in the target model which is the target of this property type mapping.

Overview relations

Name	Multiplicity	Definition
pathMapping	0..*	A relation that results in an ordered, non-unique sequence of path mappings.
combiner	0..1	A relation pointing to a result combiner.

§ 5.10 Path mapping (`PathMapping`)

A ***path mapping*** is a set of declarations which express references to data items of data objects of source data sources and actions on those.

A ***path mapping evaluation*** is the evaluation of a path mapping yielding a ***path mapping result***, which serves as input for the property mapping.

Overview attributes

Name	Multiplicity	Definition
name	0..*	The name of the path mapping.

Overview relations

Name	Multiplicity	Definition
path	1..1	A relation that points to a path.
andThen	0..*	A relation that results in an ordered, non-unique sequence of path mappings that are to be evaluated after this path mapping.

Name	Multiplicity	Definition
map	0..*	A relation that results in an ordered, non-unique sequence of result mappers.
ifMatch	0..1	A relation that points to a matcher.
ifMatchType	0..1	A relation that points to a object type ref.
repeat	0..1	A relation that points to a path repeat.

§ 5.11 Path (`Path`)

A ***path***, or path expression, is a set of declarations which, conceptually, represent a possible route through a graph of data items. A path is comprised of an ordered, non-unique sequence of segments. Each segment represents a step in that route.

A ***segment*** is a string that represents a property reference.

A ***property reference*** is a string that matches the name of a property in a data item in a source data source.

Overview attributes

Name	Multiplicity	Definition
segment	1..*	An ordered, non-unique sequence of segment strings.

§ 5.12 Path repeat (`PathRepeat`)

A ***path repeat*** is a set of declarations which describe how the path mapping evaluation should be repeated.

Overview relations

Name	Multiplicity	Definition
untilMatch	1..1	A relation that points to a matcher, which, when it matches, stops the repetition.

§ 5.13 Component (`Component`)

A ***component*** is an abstract class of things that can process the result of one or more path mapping evaluations and produce a new result.

Overview attributes

Name	Multiplicity	Definition
type	1..1	The type of the component.
options	0..1	A map (String -> Object) of options that the component can use to create results.

§ 5.13.1 Result combiner (`ResultCombiner`)

A ***result combiner*** is a component which takes as input the result of a previous element in a sequence and combines it with the current element in the sequence.

§ 5.13.2 Result mapper (`ResultMapper`)

A ***result mapper*** is a component which takes the result of a path mapping evaluation and maps it to a new result.

§ 5.13.3 Matcher (`ResultMatcher`)

A ***matcher*** is a component which takes the result of a path mapping evaluation and returns true when it matches the result and false when it does not.

§ 6. YAML IMX Model Mapping specification

A ***YAML IXM Model Mapping document*** is a concrete representation of an instance of a model mapping expressed in [*YAML*].

§ 6.1 Encoding

A YAML IXM Model Mapping document *MUST* be encoded in [*UTF-8*].

§ 6.2 Mapping definitions

§ 6.2.1 `ModelMapping`

The root node of a mapping document, the ***`ModelMapping` node*** represents the model mapping.

A ModelMapping node *MUST* express exactly one `sourceModels` key, of which the value is a mapping node of SourceModel nodes.

A ModelMapping node *MUST* express exactly one `targetModel` key, of which the value is a mapping node of TargetModel nodes.

A ModelMapping node *MUST* express zero or one `sourceRelations` key, of which the value is a sequence node of SourceRelation nodes.

A ModelMapping node *MUST* express zero or one `objectTypeMappings` key, of which the value is a mapping node of ObjectTypeMapping nodes.

> EXAMPLE 2: model mapping root node
>
> ```
> sourceModels: ...
> targetModel: ...
> sourceRelations: ...
> objectTypeMappings: ...
> ```

§ 6.2.2 `SourceModel`

A ***`SourceModel` node*** expresses a source model. It is represented by a sequence node of key/value pairs. The key of each key/value pair is the model alias of the source model. The value in the key/value pair is a ModelReference node referencing the source model definitions.

§ 6.2.3 `TargetModel`

A ***`TargetModel` node*** expresses a target model. It is represented by a key/value pair. The key of the key/value pair is the model alias of the target model. The value in the key/value pair is a ModelReference node referencing the target model definitions.

§ 6.2.4 `ModelReference`

A ***`ModelReference` node*** is represented by a mapping node. The mapping node *MUST* have:

- a key `location`, whose value is a string which expresses the file path of the mapping source file.
- a key `profile`, whose value is a string which identifies the profile of the model. The profile will be used to by the orchestration engine to load the model using the appropriate model loader.

> EXAMPLE 3: source and target model specification
>
> ```
> sourceModels:
>   - bag:
>       location: /models/bag/imbag.xml
>       profile: MIM
>   - bgt:
>       location: /models/imgeo/imgeo.xml
>       profile: MIM
>
> targetModel:
>   imx-geo:
>     location: /models/imx-geo/imx-geo.xml
>     profile: MIM
>
> objectTypeMappings: ...
> ```

6.2.5 `SourceRelation`

A ***`SourceRelation` node*** expresses a source relation mapping. It is represented by a mapping node.

It *MUST* have a `sourceType` key, whose value is an ObjectTypeRef node.

It *MUST* have a `property` key, whose value is a Property node.

6.2.6 `Property`

A ***`Property` node*** is a mapping node representing a relation property.

It *MUST* have a `name` key with a string value representing the name of the relation.

It *MUST* have a `target` key with an ObjectTypeRef node value, representing the target object type of the relation.

It *MUST* have a `multiplicity` key with a valid multiplicity string value.

It *MUST* have an `inverseName` key with a string value representing the inverse name of the relation.

It *MUST* have an `inverseMultiplicity` key with a string value representing the inverse multiplicity of the relation.

It *MUST* either have:

- a `keyMapping` key whose value is a KeyMapping node.
- a `filterMappings` key whose value is a sequence node of FilterMapping nodes.

6.2.7 `KeyMapping`

A ***`KeyMapping` node*** is a mapping node which defines the match of a source relation mapping's target object's key property, by using the name of the key properties as its keys and the names of the key value matching properties of the source relation mapping's source object as the corresponding values.

§ 6.2.8 `FilterMapping`

A ***`FilterMapping` node*** is a mapping node representing a filter mapping.

It *MUST* have a `property` key with a string value property reference, that is the value of the name of the referenced property of the source relation mapping's target object.

It *MUST* have an `operator` key whose value is a FilterOperator node.

It *MUST* have a `sourcePath` key whose value is a Path node which expresses a value of the relation the source relation mapping's source object.

§ 6.2.9 `FilterOperator`

A ***`FilterOperator` node*** is a mapping node representing a filter operator.

It *MUST* have a `type` key with a string value indicating the type of operator.

§ 6.2.10 `ObjectTypeMapping`

An ***`ObjectTypeMapping` node*** expresses an object type mapping. It is represented by a key/value pair within the mapping node that is the value of the `objectTypeMappings` key. The key in the key/value pair is the `objectTypeName` of the object type mapping. The value in the key/value pair is a sequence node that represents the set of object type mapping sets of which each item is aObjectTypeMappingSet node

> NOTE
> Note that the `mappingSets` relation from the object type mapping is implicit.

§ 6.2.11 `ObjectTypeMappingSet`

An ***`ObjectTypeMappingSet` node*** expresses an object type mapping set.

An ObjectTypeMappingSet node *MUST* have a `sourceRoot` key, which represents the source root and whose value *MUST* be an ObjecttypeRef node as a shorthand for the SourceRoot node.

An ObjectTypeMapping node *MUST* have zero or one `propertyMappings` key, whose value is a mapping node representing PropertyMapping nodes.

> EXAMPLE 4: object type mapping with object type mapping sets
>
> ```
> objectTypeMappings:
>
>   Address:
>     - sourceRoot: bag:Nummeraanduiding
>       propertyMappings: ...
>
>   Gebouw:
>     - sourceRoot: bgt:Pand
>       propertyMappings: ...
>     - sourceRoot: bgt:OverigBouwwerk
>       propertyMappings: ...
> ```

§ 6.2.12 `SourceRoot`

A ***`SourceRoot` node*** represents a source root. It is represented by a mapping node. The mapping node *MUST* have a key `type` whose value is an ObjectTypeRef node.

§ 6.2.13 `ObjectTypeRef`

An ***`ObjectTypeRef` node*** represents an object type ref. It is a generic string scalar node with the following pattern `{modelAlias}:{objectTypeName}`, where `{modelAlias}` is the model alias of a source model, and `{objectTypeName}` is the name of the object type in that source model.

§ 6.2.14 `PropertyMapping`

A ***`PropertyMapping` node*** expresses a property mapping. It is represented by a key/value pair within the mapping node that is the value of the `propertyMappings` key. The key in the key/value pair is the `propertyName` of the Property Mapping.

A PropertyMapping node *MUST* have one `pathMappings` or `pathMapping` key, whose value is a sequence node, representing PathMapping nodes, or a mapping node, representing a PathMapping

node respectively.

The evaluation of the path mapping(s) *SHOULD* yield a path mapping result that matches the type of the property in the target model which is referenced by `propertyName`.

A PropertyMapping node *MUST* have zero or one `combine` key, whose value is a ResultCombiner node.

> EXAMPLE 5: property mapping
>
> ```
> objectTypeMappings:
>   Address:
>     - sourceRoot: bag:Nummeraanduiding
>       propertyMappings:
>         addressID:
>           pathMappings:
>             - path: identificatie
>         postCode:
>           pathMapping:
>             path: postcode
> ```

§ 6.2.15 `PathMapping`

A ***`PathMapping` node*** expresses a path mapping. It is represented by a key/value pair, or a generic string scalar node, within the mapping node that is the value of the `pathMappings` key.

A ***key-value `PathMapping` node*** *MUST* have one `paths` or `path` key, whose value is a sequence node representing path expressions, or mapping node representing a path expression respectively.

A ***string scalar `PathMapping` node*** is a path expression.

A key-value PathMapping node *MUST* have zero or one `ifMatchType` key, whose value is a ObjectTypeRef node. When specified, this path mapping is only evaluated if the source root in the current property mapping evaluation matches.

A key-value PathMapping node *MUST* have zero or one `map` key, whose value is a either a sequence node of ResultMapper nodes, or a single ResultMapper node.

A key-value PathMapping node *MUST* have zero or one `andThen` key, whose value is either a sequence node of PathMapping nodes, or a single PathMapping node.

A key-value PathMapping node *MUST* have zero or one `ifMatch` key, whose value is a Matcher node.

When a PathMapping node is the value of an `andThen` key, the `ifMatch` matcher is evaluated on the path mapping result of the `andThen` bearing path mapping.

> EXAMPLE 6: path mapping
>
> ```
> objectTypeMappings:
>   Address:
>     - sourceRoot: bag:Nummeraanduiding
>       propertyMappings:
>         locatorDesignator:
>           pathMappings:
>             - path: huisnummer
>             - path: huisnummertoevoeging
>               map:
>                 type: prepend
>                 options:
>                   prefix: ' '
>             - path: huisletter
>               map:
>                 type: prepend
>                 options:
>                   prefix: ' '
>           combine:
>             type: join
>         postName:
>           pathMappings:
>             path: ligtIn/naam
>             andThen:
>               ifMatch:
>                 type: isNull
>               path: ligtAan/ligtIn/naam
> ```

§ 6.2.16 `Path`

A ***`Path` node*** expresses a path. It is represented by a mapping node, or a path expression.

A property***`MappingNode`-`Path`*** *MUST* have exactly one `expression` key, whose value is a Path expression node.

A MappingNode-Path *MUST* have zero or one `combiner` key, whose value is a ResultCombiner node.

A ***`Path` expression node*** is a generic string scalar node that expresses a path. Segments of the path are separated by a `/` character.

> ISSUE 1
> Formal description of which characters are allowed in a Path expression node.

§ 6.2.17 `Component`

A ***`Component` node*** is a mapping node which *MUST* have

- a key `type` which determines the type of the component.
- zero or one key `options`, whose value is a mapping node which defines the options for the component.

§ 6.2.18 `ResultCombiner`

c A ***`ResultCombiner` node*** is a Component node. It represents a result combiner.

§ 6.2.19 `ResultMapper`

A ***`ResultMapper` node*** is a Component node. It represents a result mapper. ggit

> EXAMPLE 7: Result mapper
>
> ```
> objectTypeMappings:
>   Address:
>     - sourceRoot: bag:Nummeraanduiding
>       propertyMappings:
>         isMainAddress:
>           pathMapping:
>             path: isHoofdadresVan/identificatie
>             map: nonNull
> ```

6.2.20 `Matcher`

A ***`Matcher` node*** is a Component node. It represents a matcher.

7. Annex-1 Mapping specifications in excel

This annex contains the mapping specifications as they are designed to map the IMX-Geo model elements to the basemodels. These mapping specifications are described in an excel spreadsheet and are the input for formulation of the machine readable YAML mapping specifications as they are developed and presented in this document. The mappings specifications furthermore are input for developing the content of the knowledge graph.

The mapping specifications consist of mapping/relating information elements from the IMX-Geo model to information elements from the models of key registers or any model that is considered and included as a source model of the IMX-Geo cross domain model.

Typically the mapping specifications follow this context chain: domain - objecttype - property - valuetype.

The excel doc containing the mapping specifications is accesible through this link: mappingspecifications

The excel contains a number of columns of which the content is explained in the excel itself.

8. Annex-2 Mapping specifications in YAML

This annex contains the mapping specifications from 7. Annex-1 Mapping specifications in excel in the form of a YAML IXM Model Mapping document.

> EXAMPLE 8: IMX Geo YAML model mapping

§ 9. Conformance

As well as sections marked as non-normative, all authoring guidelines, diagrams, examples, and notes in this specification are non-normative. Everything else in this specification is normative.

The key words *MUST* and *SHOULD* in this document are to be interpreted as described in BCP 14 [*RFC2119*] [*RFC8174*] when, and only when, they appear in all capitals, as shown here.

§ 10. List of Figures

§ A. Index

§ A.1 Terms defined by this specification

§ A.2 Terms defined by reference

§ B. References

§ B.1 Normative references

[RFC2119]
Key words for use in RFCs to Indicate Requirement Levels. S. Bradner. IETF. March 1997. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc2119

[RFC8174]

Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. B. Leiba. IETF. May 2017. Best Current Practice. URL: https://www.rfc-editor.org/rfc/rfc8174

[UTF-8]

UTF-8, a transformation format of ISO 10646. F. Yergeau. IETF. November 2003. Internet Standard. URL: https://www.rfc-editor.org/rfc/rfc3629

[YAML]

YAML Ain't Markup Language (YAML™) Version 1.2. Oren Ben-Kiki; Clark Evans; Ingy döt Net. 1 October 2009. URL: http://yaml.org/spec/1.2/spec.html

↥